

WEEK 2 연습과제

CHAPTER 02 사이킷런

▼ PART 1 사이킷런 소개와 특징

- 사이킷런(scikit-learn): 파이썬 머신러닝 라이브러리 중 가장 널리 쓰이는 라이브러리
- 매우 다양한 알고리즘과 편리한 개발 프레임워크·API 제공

▼ PART 2 붓꽃 품종 예측

- 붓꽃(Iris) 데이터로 품종 분류(Classification)
꽃잎·꽃받침의 길이/너비 피쳐(Feature)로 품종(레이블)을 예측
- 분류는 지도학습(Supervised Learning)의 대표 방법
피쳐+레이블로 모델을 학습시킨 뒤, 별도 테스트 데이터에서 미지의 레이블을 예측
- 사이킷런 모듈 명명 규칙(sklearn.*):
datasets(내장 데이터 세트) · tree(트리 기반 알고리즘) · model_selection(데이터 분리·교차검증·튜닝)
- 하이퍼 파라미터:
알고리즘별 성능을 위해 직접 입력하는 값 — 튜닝을 통해 모델 성능을 개선할 수 있음

▼ ML 모델링의 기본 프로세스 (4단계)

- ① 데이터 세트 분리 — 학습용/테스트용으로 분리 (train_test_split())
- ② 모델 학습 — 학습 데이터로 알고리즘 적용 (fit())
- ③ 예측 수행 — 학습된 모델로 테스트 데이터의 레이블 예측 (predict())
- ④ 평가 — 예측값과 실제값을 비교해 성능 측정 (예: accuracy_score()로 정확도 측정)

▼ PART 3 사이킷런의 기반 프레임워크

▼ Estimator와 fit() / predict()

- 지도학습의 모든 알고리즘 구현 클래스를 Estimator라 통칭 — 분류 구현 클래스는 Classifier, 회귀 구현 클래스는 Regressor로 지칭
- 모든 Classifier·Regressor는 학습에 fit(), 예측에 predict() 메서드를 동일하게 제공 → API 일관성이 사이킷런의 핵심 특징
- cross_val_score(), GridSearchCV 같은 평가·튜닝 클래스도 Estimator를 인자로 받아 내부에서 fit()·predict()를 호출
- 비지도학습(차원 축소·클러스터링·피쳐 추출)은 fit()+transform() 사용 — fit()은 입력 데이터 형태에 맞춰 변환 규칙을 맞추는 사전 작업(학습이 아님), transform()이 실제 변환 수행 (fit_transform()으로 한 번에 처리 가능하나 상황에 따라 개별 적용과 차이가 있어 주의 필요)

▼ 사이킷런 주요 모듈

분류	모듈명	설명
예제 데이터	datasets	내장 예제 데이터 세트 제공
피쳐 처리	preprocessing / feature_selection / feature_extraction	인코딩·스케일링 등 전처리, 피쳐 선택, 텍스트·이미지 피쳐 추출
차원 축소	decomposition	PCA·NMF·SVD 등 차원 축소 알고리즘
분리·검증·튜닝	model_selection	학습/테스트 분리, 교차 검증, 그리드 서치(하이퍼 파라미터 튜닝)

평가	metrics	Accuracy·Precision·Recall·ROC-AUC·RMSE 등 성능 지표
ML 알고리즘	ensemble / linear_model / naive_bayes / neighbors / svm / tree / cluster	양상블, 회귀(릿지·라쏘 등), 나이브 베이즈, K-NN, SVM, 결정 트리, 클러스터링 알고리즘
유틸리티	pipeline	피처 처리와 알고리즘 학습·예측을 하나로 묶어 실행

▼ 내장 예제 데이터 세트

- 별도 다운로드 없이 바로 쓸 수 있는 데이터 내장 — 분류/회귀 연습용 (load_iris·load_boston·load_breast_cancer·load_diabetes·load_digits 등)
- fetch_* 계열: 크기가 커서 최초 호출 시 인터넷에서 내려받아 저장 후 재사용 (예: fetch_covtype, fetch_20newsgroups, fetch_olivetti_faces, fetch_lfw_people/pairs, fetch_rcv1)
- 표본 데이터 생성기: make_classifications()(분류용 무작위 생성) · make_blobs()(클러스터링용 무작위 생성)
- 데이터 세트는 보통 딕셔너리형 Bunch 클래스로 반환 — 주요 키: data(피처, ndarray) · target(레이블/회귀 값, ndarray) · target_names(레이블 이름) · feature_names(피처 이름) · DESCR(데이터 세트 설명)

▼ PART 4 Model Selection 모듈 소개

- sklearn.model_selection: 학습/테스트 데이터 분리, 교차 검증, 하이퍼 파라미터 튜닝을 위한 함수·클래스 모음

▼ 학습/테스트 데이터 분리 — train_test_split()

- 학습 데이터로만 학습·예측하면 정확도가 100%로 나옴 — 이미 답을 외운 데이터로 시험을 보는 것과 같아 의미 없는 결과 (모의고사를 이미 풀어본 뒤 같은 문제로 채점하는 격)
- 따라서 예측은 반드시 학습에 쓰지 않은 전용 테스트 데이터로 수행해야 함
- 주요 파라미터: test_size(테스트 데이터 비율, 기본 0.25) · train_size(거의 안 씀) · shuffle(분리 전 데이터 섞기, 기본 True) · random_state(재현 가능한 분할을 위한 시드값)
- 반환값: (학습용 피처, 테스트용 피처, 학습용 레이블, 테스트용 레이블) 튜플

▼ 교차 검증(Cross Validation)

- 고정된 학습/테스트 데이터만으로 평가하면 과적합(Overfitting) 위험 → 테스트 데이터에만 최적화된 모델이 될 수 있음
- 교차 검증: 여러 조합의 학습/검증 데이터로 반복 학습·평가해 특정 데이터에 치우치지 않는 일반화 성능을 확보 (본고사 전 모의고사를 여러 번 보는 것에 비유)
- 데이터를 학습·검증·테스트로 세분화 → 검증 데이터는 최종 테스트 이전 모델을 다양하게 평가하는 데 사용

▼ K 폴드 교차 검증

- 데이터를 K개의 폴드로 나눠 K번 학습·검증을 반복 수행 (예: 5폴드 → 5회 평가 후 평균으로 최종 성능 산출)
- 사이킷런의 KFold 클래스로 구현 — split()이 각 반복마다 학습용/검증용 데이터의 인덱스를 반환
- 한계: 레이블 분포가 불균형(imbalanced)한 데이터에서는 특정 레이블이 학습 또는 검증 세트 어느 한쪽에 쏠리거나 아예 빠질 수 있음 (예: 극소수 레이블이 통째로 검증 세트에만 들어가면 학습이 안 되고, 반대면 검증 자체가 무의미해짐)

▼ Stratified K 폴드

- 불균형한 레이블 분포 문제를 해결하기 위한 K폴드 방식 — 원본 데이터의 레이블 분포를 먼저 고려해 학습/검증 세트에도 동일한 비율로 분배
- KFold와 사용법은 비슷하나, split() 호출 시 피쳐뿐 아니라 레이블 데이터도 반드시 함께 입력해야 함 (KFold는 레이블 없이도 동작)
- 분류(Classification)의 교차 검증은 원칙적으로 Stratified K폴드를 사용해야 함
- 회귀(Regression)는 결정값이 연속형 숫자이고 분포 개념이 성립하지 않으므로 Stratified K폴드가 지원되지 않음 (일반 K폴드 사용)

▼ cross_val_score() — 교차 검증을 한 번에

- 폴드 설정 → 반복 학습/예측 → 성능 평균까지의 과정을 한 번에 처리해주는 API
- 주요 파라미터: estimator(분류·회귀 모델) · X(피쳐) · y(레이블) · scoring(평가 지표) · cv(폴드 수)
- Classifier가 입력되면 내부적으로 Stratified K폴드 방식으로 분할 (회귀는 K폴드 방식)
- 여러 평가 지표를 동시에 확인하려면 cross_validate() 사용 (cross_val_score는 단일 지표만 반환)

▼ GridSearchCV — 교차 검증 + 하이퍼 파라미터 튜닝

- 하이퍼 파라미터: 머신러닝 알고리즘의 성능에 영향을 주는, 사용자가 직접 설정하는 값
- GridSearchCV: 여러 하이퍼 파라미터 값을 격자(Grid)처럼 순차 조합해 교차 검증 기반으로 최적 조합을 자동 탐색
- 주요 파라미터: estimator · param_grid(딕셔너리 형태로 {'파라미터명': [값 리스트]} 지정) · scoring · cv · refit
- refit=True(기본값): 찾아낸 최적 하이퍼 파라미터로 estimator를 재학습해 best_estimator_로 저장
- 결과 확인: cv_results_(조합별 상세 성능) · best_params_(최적 조합) · best_score_(최고 평균 검증 성능)
- 단점: (파라미터 조합 수 × CV 폴드 수)만큼 반복 학습해야 하므로 조합이 많을수록 수행 시간이 오래 걸림
- 일반적 흐름: GridSearchCV로 학습 데이터 내에서 최적 하이퍼 파라미터를 탐색 → 별도로 분리해둔 테스트 데이터로 최종 성능 평가

▼ PART 5 데이터 전처리

- Garbage In, Garbage Out — 데이터 전처리 품질이 ML 모델 성능에 직접적인 영향을 줌
- 결손값(NaN)은 반드시 다른 값으로 대체해야 하고, 문자열 값은 인코딩을 거쳐 숫자형으로 변환해야 함

▼ 결손값(NaN) 처리

- 피쳐에 결손값이 적으면 평균값 등으로 일괄 대체, 결손값이 대부분이면 해당 피쳐 자체를 드롭하는 것이 좋음
- 단, 전체 데이터 건수 대비 결손값 비율이 애매하면 업무 로직에 따라 신중하게 대체값을 결정해야 함 — 단순 대체가 예측을 왜곡할 수 있음

▼ 문자열 값 인코딩의 필요성

- 사이킷런 ML 알고리즘은 문자열 값을 입력값으로 허용하지 않음 — 모든 문자열 값은 인코딩되어 숫자형으로 변환되어야 함
- 문자열 피쳐는 카테고리형(코드 값) 피쳐와 텍스트형 피쳐로 구분 — 텍스트형은 피쳐 벡터화 등 별도 처리, 식별자성 문자열(단순 텍스트)은 삭제하는 것이 일반적

▼ 레이블 인코딩(Label Encoding)

- LabelEncoder 클래스로 구현 — fit()으로 인코딩 규칙을 학습하고 transform()으로 문자열 값을 숫자형 카테고리 값으로 변환
- classes_ 속성: 인코딩 전 원본 문자열 값 목록 확인 가능 / inverse_transform(): 인코딩된 숫자값을 원래 문자열 값으로 복원
- 한계: 숫자 값의 크기 작음이라는 특성이 부여돼 일부 ML 알고리즘(선형 회귀 등)에서는 예측 성능이 왜곡될 수 있음 — 트리 계열 알고리즘은 영향받지 않음

▼ 원-핫 인코딩(One-Hot Encoding)

- 레이블 인코딩의 숫자 크기 문제를 해결하기 위한 방식 — 피쳐 값의 고유 유형마다 새로운 칼럼을 만들어, 해당하는 칼럼에만 1을 표시하고 나머지는 0으로 표시
- OneHotEncoder 클래스: 입력값이 2차원 데이터여야 하며, 적용 전 문자열 값을 먼저 숫자형으로 변환(LabelEncoder)해야 함 — 변환 결과는 희소 행렬(Sparse Matrix)이라 toarray()로 밀집 행렬로 변환해 확인
- 판다스 get_dummies(): 문자열 카테고리 값을 숫자형 사전 변환 없이 바로 원-핫 인코딩해주는 더 간편한 방법

▼ 피쳐 스케일링과 정규화

- 서로 다른 변수(피쳐)의 값 범위를 일정한 수준으로 맞추는 작업

- 표준화(Standardization): 평균이 0이고 분산이 1인 가우시안 정규분포 형태로 값을 변환 (수식: $(x_i - \text{평균}) / \text{표준편차}$)
- 정규화(Normalization): 서로 다른 피처의 크기를 통일하기 위해 최소 0 ~ 최대 1 사이 값으로 변환 (수식: $(x_i - \text{최소값}) / (\text{최대값} - \text{최소값})$)
- 사이킷런의 Normalizer 모듈은 위 개념과 다르게, 개별 벡터(row)의 크기를 1로 맞추기 위해 각 피처 값을 벡터 크기로 나누는 선형대수 방식의 정규화 — 용어 혼동 주의

▼ StandardScaler / MinMaxScaler

- StandardScaler: 표준화를 지원하는 클래스 — 서포트벡터머신·선형회귀·로지스틱회귀는 데이터가 가우시안 분포를 따른다고 가정하고 구현돼, 사전 표준화가 예측 성능 향상에 중요
- MinMaxScaler: 데이터 값을 0과 1 사이 범위로 변환(음수 값이 있으면 -1~1) — 데이터 분포가 가우시안 분포가 아닐 때 적용을 고려

▼ 학습·테스트 데이터 스케일링 시 유의점

- fit()은 학습 데이터에만 적용하고, 테스트 데이터는 그 fit()으로 학습된 Scaler의 transform()만 사용해야 함
- 테스트 데이터에 다시 fit()(또는 fit_transform())을 적용하면 학습 데이터와 테스트 데이터의 스케일링 기준이 달라져 예측 결과가 왜곡됨
- 가급적 전체 데이터를 먼저 스케일링 변환한 뒤 학습/테스트 데이터로 분리하는 것이 바람직 (불가피하게 나눠 처리할 경우 테스트 데이터에는 반드시 transform()만 적용)

▼ PART 7 정리

- 사이킷런: 파이썬 계열 대표 머신러닝 패키지 — 다양한 알고리즘, 직관적인 API 프레임워크, 편리한 다양한 모듈 지원이 강점
- 머신러닝 애플리케이션의 기본 구성: 데이터 전처리 → 학습/테스트 데이터 분리 → 모델 학습 → 예측 → 실제값과의 비교를 통한 평가
- 데이터 전처리
 - 결손값(Null) 처리, 레이블 인코딩·원-핫 인코딩 같은 인코딩 작업, 피처의 스케일링/정규화 작업 등으로 구성 — ML 알고리즘이 최적으로 동작하도록 데이터를 사전 정리하는 과정
- 교차 검증의 필요성
 - 학습된 모델은 반드시 별도의 테스트 데이터로 평가해야 함
 - 테스트 데이터 건수가 부족하거나 고정된 테스트 세트로 반복 평가하면, 해당 테스트 데이터에만 치우친 빈약한 모델이 될 위험이 있음
 - 이를 보완하기 위해 학습 데이터를 다시 학습·검증 데이터로 나눈 여러 폴드 세트로 교차 검증 수행 — 사이킷런은 KFold·StratifiedKFold·cross_val_score() 등을 지원
 - GridSearchCV: 교차 검증을 통해 최적의 하이퍼 파라미터를 함께 추출해주는 API

CHAPTER 03 평가

▼ PART 0 개요

- 머신러닝은 데이터 가공·변환 → 모델 학습·예측 → 평가(Evaluation)의 프로세스로 구성 — 성능 평가 지표는 분류·회귀에 따라 다르며, 회귀는 대부분 실제값-예측값의 오차 평균에 기반
- 분류의 평가 방법은 예측 결과와 실제 결과가 얼마나 정확한지에 기반하지만, 단순 정확도만으로 판단하면 잘못된 평가에 빠질 수 있음 — 특히 0/1 이진 분류에서는 정확도보다 다른 지표가 더 중요한 경우가 많음
- 분류 성능 평가 지표: 정확도(Accuracy) · 오차행렬(Confusion Matrix) · 정밀도(Precision) · 재현율(Recall) · F1 스코어 · ROC AUC

▼ PART 1 정확도(Accuracy)

- 정확도(Accuracy) = 예측 결과가 실제 값과 동일한 데이터 건수 / 전체 예측 데이터 건수

- 직관적으로 모델 성능을 나타내는 지표이지만, 데이터의 구성(레이블 분포)에 따라 성능이 왜곡될 수 있어 정확도 수치 하나만으로 성능을 평가해서는 안 됨
- 예시(타이타닉): 별다른 알고리즘 없이 성별 조건 하나만으로 생존 여부를 예측해도 정확도가 80%대로 높게 나올 수 있음
- 특히 불균형한(imbalanced) 레이블 데이터 세트에서 문제가 큼 — 다수 클래스로만 예측해도 정확도가 매우 높게 나옴 (예: MNIST를 '숫자 7인지 아닌지'의 이진 분류로 바꾼 뒤, 무조건 '아니다(0)'로만 예측해도 정확도 약 90%)
- 결론: 정확도는 분류 모델의 성능을 측정하는 여러 요소 중 하나일 뿐 — 불균형한 데이터 세트에서는 정확도 단독 사용에 특히 주의해야 함

▼ PART 2 오차 행렬(Confusion Matrix)

- 이진 분류에서 잘 활용되는 성능 지표 — 학습된 분류 모델이 예측을 수행하면서 얼마나 헛갈리고(confused) 있는지, 어떤 유형의 오류를 내는지 함께 나타냄
- 실제 클래스 값과 예측 클래스 값의 조합에 따라 TN·FP·FN·TP 4분면으로 구성 — 앞 글자 True/False는 예측과 실제값의 일치 여부, 뒤 글자 Negative/Positive는 예측 결과값(0/1)을 의미

구분	예측 Negative(0)	예측 Positive(1)
실제 Negative(0)	TN (True Negative) — 예측·실제 모두 Negative	FP (False Positive) — Positive로 예측했으나 실제로는 Negative
실제 Positive(1)	FN (False Negative) — Negative로 예측했으나 실제로는 Positive	TP (True Positive) — 예측·실제 모두 Positive

- 사이킷런 confusion_matrix() API로 산출 — 이진 분류에서 반환된 ndarray는 [0,0]=TN, [0,1]=FP, [1,0]=FN, [1,1]=TP 위치에 대응
- 정확도도 오차 행렬 값으로 재정의 가능: $\text{정확도} = (TN + TP) / (TN + FP + FN + TP)$
- 불균형한 이진 분류에서는 상대적으로 적고 중점적으로 찾아야 하는 값에 Positive(1)를, 그렇지 않은 값에 Negative(0)를 부여하는 경우가 일반적 (예: 사기 탐지 — 사기 행위 Positive, 정상 행위 Negative / 암 진단 — 양성 Positive, 음성 Negative)
- 불균형 데이터에서는 Positive 건수가 매우 적어 Negative로 예측하는 경향이 강해지고, 그 결과 TN은 매우 커지고 TP·FP·FN은 작아짐 — 정확도 지표가 Positive에 대한 예측 정확도를 제대로 반영하지 못한 채 수치적으로 매우 높게 나타나는 판단 오류를 일으킴
- 따라서 정확도만으로는 신뢰하기 어려우며, 이를 보완하는 정밀도(Precision)·재현율(Recall) 같은 지표가 필요함

▼ PART 3 정밀도와 재현율

- 정밀도(Precision)·재현율(Recall)은 Positive 데이터 세트의 예측 성능에 좀 더 초점을 맞춘 평가 지표
- 정밀도 = $TP / (FP + TP)$ — Positive로 예측한 것 중 실제로도 Positive인 데이터의 비율 (양성 예측도)
- 재현율 = $TP / (FN + TP)$ — 실제 값이 Positive인 것 중 Positive로 예측한 데이터의 비율 (민감도 · TPR로도 불림)
- 업무 특성에 따른 중요도
 - 재현율이 상대적으로 더 중요한 경우: 실제 Positive 데이터를 Negative로 잘못 판단하면 업무상 큰 영향이 발생하는 경우 (예: 암 진단 — 암을 놓치면 치명적이나, 정상을 암으로 오판해도 재검사 수준의 비용 / 금융 사기 탐지 — 사기를 놓치면 손해가 크나, 정상 거래를 사기로 오판해도 재확인 절차로 해결 가능)
 - 정밀도가 상대적으로 더 중요한 경우: 실제 Negative 데이터를 Positive로 잘못 판단하면 업무상 영향이 큰 경우 (예: 스팸 메일 필터링 — 정상 메일을 스팸으로 잘못 분류하면 아예 받지 못해 업무 차질, 스팸을 걸러내지 못하는 것은 상대적으로 불편한 정도)
 - 재현율(FN 최소화 초점)과 정밀도(FP 최소화 초점)는 둘 다 TP를 높인다는 점은 같지만 초점이 달라 서로 보완적인 지표 — 가장 좋은 성능 평가는 둘 다 높은 수치를 얻는 것이며, 한쪽만 매우 높고 다른 한쪽이 매우 낮은 경우는 바람직하지 않음
- 정밀도/재현율 트레이드오프
 - 분류 결정 임계값(Threshold)을 조정하면 정밀도 또는 재현율의 수치를 상호 보완적으로 조정할 수 있음
 - 사이킷런 분류 알고리즘은 개별 데이터가 특정 레이블에 속할 결정 확률을 구한 뒤, 확률이 큰 레이블로 예측 — 이진 분류의 기본 임계값은 0.5이며, predict_proba()가 레이블별 예측 확률을 반환하고 predict()는 이 확률이 임계값보다 큰 클래스로 최종 결정

- 사이킷런 Binarizer 클래스로 임계값을 조정할 예측값을 직접 구현 가능 — 입력값이 지정한 threshold보다 작거나 같으면 0, 크면 1로 변환
- 임계값을 낮추면 Positive로 예측할 확률이 높아져 재현율은 올라가고 정밀도는 떨어짐 / 임계값을 높이면 반대로 정밀도가 올라가고 재현율은 떨어짐
- precision_recall_curve() API로 임계값 변화에 따른 정밀도·재현율 값을 한 번에 구하고, 곡선 형태의 그래프로 시각화할 수 있음
- **정밀도와 재현율의 맹점**
 - 정밀도·재현율 수치는 업무 환경에 맞게 두 지표를 상호 보완할 수 있는 수준에서 조정해야 하며, 단순히 하나의 지표 수치만 극단적으로 높이는 수단으로 사용해서는 안 됨
 - 정밀도를 100%로 만드는 방법: 확실한 기준을 가진 데이터 단 1건만 Positive로 예측하고 나머지는 모두 Negative로 예측 — FP가 0이 되어 정밀도($=TP/(TP+FP)$)는 100%가 되지만 실질적 의미는 없음
 - 재현율을 100%로 만드는 방법: 모든 데이터를 Positive로 예측 — FN이 0이 되어 재현율($=TP/(TP+FN)$)은 100%가 되지만 이 역시 숫자 놀음에 불과
 - 이처럼 한쪽 지표만 참조하면 극단적인 수치 조작이 가능하므로, 정밀도와 재현율을 적절히 결합한 종합적인 평가 지표가 필요함 — F1 스코어

▼ PART 4 F1 스코어

- F1 스코어(Score): 정밀도와 재현율을 결합한 지표 — 정밀도와 재현율이 어느 한쪽으로 치우치지 않을 때 상대적으로 높은 값을 가짐 (정밀도·재현율의 조화평균 형태)
- 공식: $F1 = 2 / (1/recall + 1/precision) = 2 \times (precision \times recall) / (precision + recall)$
- 예시: 모델 A(정밀도 0.9·재현율 0.1로 극단적 차이)의 F1은 0.18인 반면, 모델 B(정밀도 0.5·재현율 0.5로 차이 없음)의 F1은 0.5 — 두 지표 값이 비슷할수록 B처럼 더 높은 F1 스코어를 가짐
- 사이킷런은 F1 스코어 산출을 위해 f1_score() API를 제공

▼ PART 5 ROC 곡선과 AUC

- ROC 곡선(Receiver Operation Characteristic Curve, 수신자 판단 곡선): 이진 분류의 예측 성능 측정에서 중요하게 사용되는 지표 — 원래 통신 장비 성능 평가용으로 고안돼 의학 분야에서 많이 쓰이지만, 머신러닝 이진 분류 모델 평가에도 중요한 지표
- FPR(False Positive Rate)이 변할 때 TPR(True Positive Rate)이 어떻게 변하는지를 나타내는 곡선 — X축 FPR, Y축 TPR
- TPR은 재현율과 동일($TP/(FN+TP)$), 민감도(Sensitivity)로도 불림 — 실제 Positive가 정확히 예측돼야 하는 수준을 나타냄
- TNR(True Negative Rate, 특이성 Specificity) = $TN/(FP+TN)$ — 실제 Negative가 정확히 예측돼야 하는 수준 / $FPR = FP/(FP+TN) = 1 - TNR = 1 - \text{특이성}$
- ROC 곡선은 분류 결정 임계값을 1부터 0까지 변화시키며 FPR과 그에 따른 TPR 값을 구해 그린 곡선 — 임계값이 1에 가까우면 FPR·TPR 모두 0에 가깝고, 임계값이 0에 가까우면 모두 1에 가까워짐
- 가운데 대각선 직선은 랜덤 수준(동전 던지기 수준)의 이진 분류 ROC 직선으로 AUC 값 0.5에 해당 — 곡선이 대각선에서 멀어지고 왼쪽 상단 모서리로 가파르게(직사각형에 가깝게) 이동할수록 성능이 뛰어남
- AUC(Area Under Curve): ROC 곡선 밑의 면적 값 — 일반적으로 1에 가까울수록 좋은 수치이며, 보통의 분류 모델은 0.5 이상의 AUC 값을 가짐
- 사이킷런은 roc_curve() API로 FPR·TPR·임계값을, roc_auc_score() API로 AUC 값을 산출

▼ PART 7 정리

- 이진 분류의 레이블 값이 불균형하게 분포된 경우(0이 매우 많거나 그 반대), 단순히 예측 결과와 실제 결과가 일치하는지만 보는 정확도만으로는 모델 성능을 제대로 평가할 수 없음
- 오차 행렬은 실제 클래스 값과 예측 클래스 값이 True/False에 따라 매핑되는 4분면 행렬을 기반으로 예측 성능을 평가 — 정확도·정밀도·재현율 수치는 모두 TN·FP·FN·TP 값의 조합으로 만들어짐
- 정밀도(Precision)와 재현율(Recall)은 Positive 데이터 세트의 예측 성능에 좀 더 초점을 맞춘 평가 지표 — 업무 특성에 따라 재현율이 더 중요한 경우(암 진단 등)와 정밀도가 더 중요한 경우(스팸 필터링 등)가 있으며, 분류 결정 임계값(Threshold) 조정으로 정밀도 또는 재현율의 수치를 상호 보완적으로 조정할 수 있음

- F1 스코어는 정밀도와 재현율을 결합한 평가 지표이며, 정밀도와 재현율이 어느 한쪽으로 치우치지 않을 때 높은 지표값을 가짐
- ROC-AUC는 이진 분류의 성능 평가를 위해 일반적으로 가장 많이 사용되는 지표 — AUC(Area Under Curve) 값은 ROC 곡선 밑의 면적을 구한 것으로서 일반적으로 1에 가까울수록 좋은 수치
- 분류 평가 지표를 배웠으니, 다음 장에서는 머신러닝 기반의 분류를 본격적으로 구현